

Métodos Computacionais em Física de Altas Energias

Interação entre linguagens

Marco Dias

4 de setembro de 2026

- C++ e ROOT são muito próximos e fáceis de combinar
- Classes e bibliotecas ROOT disponíveis após instalação
- No Makefile: incluir caminho e linkar bibliotecas `-l$(ROOTSYS)/include`
- Em tempo de execução: `LD_LIBRARY_PATH` deve conter o caminho das bibliotecas ROOT

Por que compilar código ROOT?

- 1 Programas de teste com ROOT podem ser escritos
- 2 Velocidade: macros ROOT são mais lentas que código compilado
- 3 Construções de linguagem não suportadas pelo CINT ficam disponíveis

Exemplo: Compilando um script com TGraph

Modificações necessárias:

- Mudar void myGraphPlotting() para int main()
- Incluir cabeçalhos necessários
- Adicionar Makefile

```
1 #include <iostream>
2 #include "TGraph.h"
3 #include "TCanvas.h"
4 #include "TH2F.h"
5 #include "TLatex.h"
6 using namespace std;
7 int main() {
8     // Código de plotagem aqui
9     return 0;
10 }
```

Makefile para ROOT

```
1 CXX = g++
2 CXXFLAGS = -O2 -Wall
3 ROOTCFLAGS = $(shell root-config --cflags)
4 ROOTLIBS = $(shell root-config --glibs)
5
6 myPlot: myPlot.o
7     $(CXX) -o $@ $^ $(ROOTLIBS)
8
9 myPlot.o: myPlot.cpp
10    $(CXX) $(CXXFLAGS) $(ROOTCFLAGS) -c $<
```

Listing 1: Exemplo de Makefile

Compilando com ACLiC

- ACLiC = Automatic Library Compiler for CINT
- Não precisa de Makefile
- Nome da função não precisa ser alterado

```
1 root [] .L myScript.C++
```

- Cria biblioteca compartilhada myScript_C.so
- Função fica disponível:

```
1 root [] myScript()
```

- Sempre incluir #include nos scripts
- Não usar extensões do CINT
- Programar evitando limitações do CINT

Rootificação de uma Classe

Para integrar uma classe ao ROOT:

- 1 Herdar de `TObject`
- 2 Adicionar macro `ClassDef(MyClass,1)` na definição
- 3 Adicionar macro `ClassImp(MyClass)` na implementação
- 4 Fornecer construtor padrão

Classe `TObject`

- Fornece comportamento padrão para objetos ROOT
- Interface primária para:
 - E/S (I/O)
 - Tratamento de erros
 - Desenho
- Sem herdar de `TObject`: recursos de E/S não disponíveis

Rootificação de uma Classe

```
1 #ifndef MYPARTICLE_H
2 #define MYPARTICLE_H
3
4 #include "TObject.h"
5 #include "TLorentzVector.h"
6 #include <vector>
7
8 class MyParticle : public TObject {
9 private:
10     // Dados da partícula
11     TLorentzVector fP4;           // Quadrimomento (E, px, py, pz)
12     Int_t fCharge;               // Carga elétrica em unidades de e
13     Int_t fPdgId;               // ID PDG da partícula
14     Float_t fIsolation;         // Isolamento (0=isolado, 1=
15     overlapping)
16     Float_t fChi2;              // Chi2 do ajuste da trajetória
17     std::vector<Float_t> fHits;  // Depósitos de energia no
```

Rootificação de uma Classe Definição da Classe

```
1 public:
2     // Construtor padrão - OBRIGATÓRIO para ROOT I/O
3     MyParticle();
4     // Construtor com parâmetros
5     MyParticle(Double_t energy, Double_t px, Double_t py,
6     Double_t pz, Int_t charge, Int_t pdgId);
7     // Destrutor
8     virtual ~MyParticle() {}
9     // Métodos Getter
10    Double_t GetEnergy() const { return fP4.Energy(); }
11    Double_t GetPt() const { return fP4.Pt(); }
12    Double_t GetEta() const { return fP4.Eta(); }
13    Double_t GetPhi() const { return fP4.Phi(); }
14    Double_t GetMass() const { return fP4.Mag(); }
15    Int_t GetCharge() const { return fCharge; }
16    Int_t GetPdgId() const { return fPdgId; }
17    Float_t GetIsolation() const { return fIsolation; }
```

Rootificação de uma Classe - Definição da Classe

```
1  Float_t GetChi2() const { return fChi2; }
2  Int_t GetNHits() const { return fHits.size(); }
3  // Métodos Setter
4  void SetP4(Double_t energy, Double_t px, Double_t py, Double_t
    pz);
5  void SetCharge(Int_t charge) { fCharge = charge; }
6  void SetPdgId(Int_t pdgId) { fPdgId = pdgId; }
7  void SetIsolation(Float_t iso) { fIsolation = iso; }
8  void AddHit(Float_t energy) { fHits.push_back(energy); }
9  // Método para impressão
10 void Print(Option_t* option = "") const;
11 // Macro ClassDef - ESSENCIAL para ROOT I/O
12 // 0 número 1 é a versão da classe
13 ClassDef(MyParticle, 1)
14 };
15 #endif
```

Listing 3: MyParticle.h - Definição da Classe

Rootificação - Implementação da Classe

```
1 #include "MyParticle.h"
2 #include <iostream>
3 #include <cmath>
4 // Macro ClassImp - NECESSÁRIA para a implementação
5 ClassImp(MyParticle)
6 // Construtor padrão - DEVE inicializar todos os membros
7 MyParticle::MyParticle() :
8     fP4(0, 0, 0, 0), // TLorentzVector(px, py, pz, E)
9     fCharge(0),
10    fPdgId(0),
11    fIsolation(0.0),
12    fChi2(0.0) {
13    // fHits é inicializado automaticamente como vazio
14 }
```

Rootificação - Implementação da Classe

```
1 // Construtor com parâmetros
2 MyParticle::MyParticle(Double_t energy, Double_t px, Double_t py,
3   Double_t pz, Int_t charge, Int_t pdgId) :
4     fP4(px, py, pz, energy),
5     fCharge(charge),
6     fPdgId(pdgId),
7     fIsolation(0.0),
8     fChi2(0.0) {} // fHits inicializado vazio
9 void MyParticle::SetP4(Double_t energy, Double_t px, Double_t py,
10   Double_t pz) {
11     fP4.SetPx(px);
12     fP4.SetPy(py);
13     fP4.SetPz(pz);
14     fP4.SetE(energy);
15 }
```

Rootificação - Implementação da Classe

```
1 void MyParticle::Print(Option_t* option) const {
2     std::cout << "=== MyParticle ===" << std::endl;
3     std::cout << "PDG ID: " << fPdgId << std::endl;
4     std::cout << "Charge: " << fCharge << " e" << std::endl;
5     std::cout << "Energy: " << GetEnergy() << " GeV" << std::endl;
6     std::cout << "Pt: " << GetPt() << " GeV/c" << std::endl;
7     std::cout << "Eta: " << GetEta() << std::endl;
8     std::cout << "Phi: " << GetPhi() << " rad" << std::endl;
9     std::cout << "Mass: " << GetMass() << " GeV/c^2" << std::endl;
10    std::cout << "Isolation: " << fIsolation << std::endl;
11    std::cout << "Chi2: " << fChi2 << std::endl;
12    std::cout << "Number of hits: " << fHits.size() << std::endl;
13 }
```

Listing 4: MyParticle.cc - Implementação

Rootificação - Classe MyEvent

```
1 #ifndef MYEVENT_H
2 #define MYEVENT_H
3
4 #include "TObject.h"
5 #include "MyParticle.h"
6 #include <vector>
7 #include <string>
8
9 class MyEvent : public TObject {
10 private:
11     Int_t fRunNumber;           // Número do run
12     Int_t fEventNumber;        // Número do evento
13     Int_t fLuminosityBlock;    // Bloco de luminosidade
14     Float_t fEventWeight;      // Peso do evento
15     std::string fProcessType;  // Tipo de processo físico
```

Rootificação - Classe MyEvent

```
1 // Coleções de partículas
2 std::vector<MyParticle> fMuons;
3 std::vector<MyParticle> fElectrons;
4 std::vector<MyParticle> fJets;
5 std::vector<MyParticle> fPhotons;
6 std::vector<MyParticle> fTaus;
7 // Metadados
8 Float_t fMissingET;           // Energia faltante (MET)
9 Float_t fMissingETPhi;       // Phi do MET
10 public:
11 // Construtor padrão - OBRIGATÓRIO
12 MyEvent();
13 // Construtor com parâmetros básicos
14 MyEvent(Int_t run, Int_t event, Int_t lumi);
15 // Destrutor
16 virtual ~MyEvent() {}
```

Rootificação - Classe MyEvent

```
1 // Getters
2 Int_t GetRunNumber() const { return fRunNumber; }
3 Int_t GetEventNumber() const { return fEventNumber; }
4 Int_t GetLuminosityBlock() const { return fLuminosityBlock; }
5 Float_t GetEventWeight() const { return fEventWeight; }
6 Float_t GetMissingET() const { return fMissingET; }
7 Int_t GetNMuons() const { return fMuons.size(); }
8 Int_t GetNElectrons() const { return fElectrons.size(); }
9 Int_t GetNJets() const { return fJets.size(); }
10 Int_t GetNPhotons() const { return fPhotons.size(); }
11
12 // Setters
13 void SetRunNumber(Int_t run) { fRunNumber = run; }
14 void SetEventNumber(Int_t event) { fEventNumber = event; }
15 void SetMissingET(Float_t met, Float_t phi);
```

Rootificação - Classe MyEvent

```
1 // Métodos para adicionar partículas
2 void AddMuon(const MyParticle& mu) { fMuons.push_back(mu); }
3 void AddElectron(const MyParticle& el) { fElectrons.push_back(el
4 ); }
5 void AddJet(const MyParticle& jet) { fJets.push_back(jet); }
6 void AddPhoton(const MyParticle& photon) { fPhotons.push_back(
7 photon); }
8
9 // Métodos para acessar coleções
10 const std::vector<MyParticle>& GetMuons() const { return fMuons;
11 }
12 const std::vector<MyParticle>& GetElectrons() const { return
13 fElectrons; }
14 const std::vector<MyParticle>& GetJets() const { return fJets; }
15
16 // Método para impressão resumida
17 void PrintSummary(Option_t* option = "") const;
```

```
1 // Macro ClassDef
2 ClassDef(MyEvent, 1)
3 };
4 #endif
```

Listing 5: MyEvent.h - Classe que contém várias partículas

Rootificação - Implementação do MyEvent

```
1 #include "MyEvent.h"
2 #include <iostream>
3
4 ClassImp(MyEvent)
5
6 MyEvent::MyEvent() :
7     fRunNumber(0),
8     fEventNumber(0),
9     fLuminosityBlock(0),
10    fEventWeight(1.0),
11    fProcessType("Unknown"),
12    fMissingET(0.0),
13    fMissingETPhi(0.0) {
14    // Todas as coleções são inicializadas vazias
15 }
```

Rootificação - Implementação do MyEvent

```
1
2 MyEvent::MyEvent(Int_t run, Int_t event, Int_t lumi) :
3     fRunNumber(run),
4     fEventNumber(event),
5     fLuminosityBlock(lumi),
6     fEventWeight(1.0),
7     fProcessType("Unknown"),
8     fMissingET(0.0),
9     fMissingETPhi(0.0) {
10 }
11
12 void MyEvent::SetMissingET(Float_t met, Float_t phi) {
13     fMissingET = met;
14     fMissingETPhi = phi;
15 }
```

Rootificação - Implementação do MyEvent

```
1 void MyEvent::PrintSummary(Option_t* option) const {
2     std::cout << "=== MyEvent ===" << std::endl;
3     std::cout << "Run: " << fRunNumber << std::endl;
4     std::cout << "Event: " << fEventNumber << std::endl;
5     std::cout << "Luminosity Block: " << fLuminosityBlock << std::
        endl;
6     std::cout << "Process: " << fProcessType << std::endl;
7     std::cout << "Event Weight: " << fEventWeight << std::endl;
8     std::cout << "MET: " << fMissingET << " GeV, phi: " <<
        fMissingETPhi << " rad" << std::endl;
9     std::cout << "Muons: " << fMuons.size() << std::endl;
10    std::cout << "Electrons: " << fElectrons.size() << std::endl;
11    std::cout << "Jets: " << fJets.size() << std::endl;
12    std::cout << "Photons: " << fPhotons.size() << std::endl;
13 }
```

Listing 6: MyEvent.cc - Implementação

Rootificação - LinkDef.h

```
1 #ifdef __CINT__
2 // Desativa links para tudo por padrão
3 #pragma link off all globals;
4 #pragma link off all classes;
5 #pragma link off all functions;
6 // Ativa links para nossas classes
7 #pragma link C++ class MyParticle+;
8 #pragma link C++ class vector<MyParticle>;
9 #pragma link C++ class MyEvent+;
10 #pragma link C++ class vector<MyEvent>;
11 // Se quisermos também disponibilizar os métodos
12 // #pragma link C++ function MyParticle::Print;
13 #endif
```

Listing 7: LinkDef.h - Diz ao rootcint o que exportar

O + indica para usar o novo sistema de E/S

Rootificação - Makefile Completo

```
1 # Compilador e flags
2 CXX = g++
3 CXXFLAGS = -O2 -Wall -fPIC
4 ROOTCFLAGS = $(shell root-config --cflags)
5 ROOTLIBS = $(shell root-config --glibs)
6 INCLUDES = -I$(ROOTSYS)/include
7
8 # Fontes e objetos
9 CLASS_SRCS = MyParticle.cc MyEvent.cc
10 CLASS_OBJS = $(CLASS_SRCS:.cc=.o)
11 DICT_SRCS = eventdict.cc
12 DICT_OBJS = $(DICT_SRCS:.cc=.o)
13 OBJS = $(CLASS_OBJS) $(DICT_OBJS)
14 LIB = libMyEvent.so
15
16 # Alvo principal
17 all: $(LIB)
```

Rootificação - Makefile Completo

```
1 # Gerar dicionário
2 $(DICT_SRCS): MyParticle.h MyEvent.h LinkDef.h
3     rootcint -f $@ -c $(INCLUDES) MyParticle.h MyEvent.h LinkDef.h
4
5 # Compilar objetos
6 %.o: %.cc
7     $(CXX) $(CXXFLAGS) $(ROOTCFLAGS) $(INCLUDES) -c $<
8
9 # Compilar dicionário
10 $(DICT_OBJS): $(DICT_SRCS)
11     $(CXX) $(CXXFLAGS) $(ROOTCFLAGS) $(INCLUDES) -c $<
12
13 # Criar biblioteca compartilhada
14 $(LIB): $(OBJS)
15     $(CXX) -shared -o $@ $^ $(ROOTLIBS)
```

```
1 # Limpar
2 clean:
3     rm -f $(OBJS) $(LIB) $(DICT_SRCS) eventdict.h
4
5 # Limpar tudo incluindo arquivos temporários
6 distclean: clean
7     rm -f *~ *.d
```

Listing 8: Makefile - Compilação

Rootificação - Usando no ROOT

```
1 // Carregar a biblioteca
2 root [] .L libMyEvent.so
3 // Criar uma partícula
4 root [] MyParticle *muon = new MyParticle(100.0, 10.0, 5.0, 1.0, -1,
      13)
5 root [] muon->Print()
6 // Criar um evento
7 root [] MyEvent *event = new MyEvent(1, 12345, 50)
8 root [] event->AddMuon(*muon)
9 root [] event->AddElectron(MyParticle(50.0, 5.0, 2.0, 0.5, -1, 11))
10 // Verificar o evento
11 root [] event->PrintSummary()
12 root [] event->GetNMuons()
13 (Int_t)1
```

```
1 // Salvar em arquivo ROOT
2 root [] TFile f("events.root", "RECREATE")
3 root [] event->Write("event1")
4 root [] f.Close()
5 // Ler de volta
6 root [] TFile f2("events.root")
7 root [] MyEvent *event2 = (MyEvent*)f2.Get("event1")
8 root [] event2->PrintSummary()
```

Listing 9: Exemplo de uso no ROOT

- Necessários para acesso via interpretador
- Criados com rootcint (ou rootcling)
- Geram métodos:
 - Streamer()
 - TBuffer &operator»()
 - ShowMembers()

Arquivo LinkDef.h

- Diz ao rootcint para quais classes gerar stubs
- Nome DEVE conter LinkDef.h ou linkdef.h
- Deve ser o último argumento na linha de comando

```
1 #ifdef __CINT__  
2 #pragma link off all globals;  
3 #pragma link off all classes;  
4 #pragma link off all functions;  
5  
6 #pragma link C++ class MyJet+;  
7 #pragma link C++ class vector<MyJet>;  
8 #pragma link C++ class MyEvent+;  
9 #endif
```

Listing 10: Exemplo de LinkDef.h

```
1 rootcint -f eventdict.cc -c -I. MyEvent.h MyJet.h LinkDef.h
```

Listing 11: Comando rootcint

- eventdict.cc: arquivo do dicionário gerado
- -I.: caminho de inclusão
- Arquivos de cabeçalho das classes
- LinkDef.h (último argumento)

Exemplo: Classe MyParticle (1/4)

Vamos criar uma classe para representar partículas em análise de colisões.

```
1 #ifndef MYPARTICLE_H
2 #define MYPARTICLE_H
3
4 #include "TObject.h"
5 #include "TLorentzVector.h"
6 #include <vector>
7
8 class MyParticle : public TObject {
9 private:
10     TLorentzVector fP4;           // Quadrimomento
11     Int_t fCharge;                // Carga elétrica
12     Int_t fPdgId;                // ID PDG
13     Float_t fIsolation;          // Isolamento
14     Float_t fChi2;               // Qualidade do ajuste
```

Exemplo: Classe MyParticle (1/4)

```
1 public:
2     MyParticle();                // Construtor padrão
3     MyParticle(Double_t E, Double_t px, Double_t py, Double_t pz);
4
5     // Getters
6     Double_t GetEnergy() const { return fP4.Energy(); }
7     Double_t GetPt() const { return fP4.Pt(); }
8     Double_t GetEta() const { return fP4.Eta(); }
9     Double_t GetPhi() const { return fP4.Phi(); }
10
11     ClassDef(MyParticle, 1)
12 };
13 #endif
```

Listing 12: MyParticle.h

Exemplo: Classe MyParticle (2/4)

Implementação da classe.

```
1 #include "MyParticle.h"
2
3 ClassImp(MyParticle)
4
5 MyParticle::MyParticle() : fCharge(0), fPdgId(0),
6                           fIsolation(0), fChi2(0) {
7     // Construtor padrão - OBRIGATÓRIO
8 }
9
10 MyParticle::MyParticle(Double_t E, Double_t px,
11                        Double_t py, Double_t pz)
12     : fP4(px, py, pz, E), fCharge(0), fPdgId(0),
13       fIsolation(0), fChi2(0) {
14     // Construtor com parâmetros
15 }
```

Listing 13: MyParticle.cc

Exemplo: Classe MyParticle (3/4)

Definição da classe MyEvent que contém várias partículas.

```
1 #ifndef MYEVENT_H
2 #define MYEVENT_H
3
4 #include "TObject.h"
5 #include "MyParticle.h"
6 #include <vector>
7
8 class MyEvent : public TObject {
9 private:
10     Int_t fRunNumber;
11     Int_t fEventNumber;
12     Int_t fLuminosityBlock;
13     std::vector<MyParticle> fMuons;
14     std::vector<MyParticle> fElectrons;
15     std::vector<MyParticle> fJets;
```

Exemplo: Classe MyParticle (3/4)

```
1 public:
2     MyEvent();
3     void AddMuon(const MyParticle& mu);
4     void AddElectron(const MyParticle& el);
5     void AddJet(const MyParticle& jet);
6
7     // Getters
8     Int_t GetRunNumber() const { return fRunNumber; }
9     Int_t GetEventNumber() const { return fEventNumber; }
10
11     ClassDef(MyEvent, 1)
12 };
13 #endif
```

Listing 14: MyEvent.h

Exemplo: Classe MyEvent (Implementação)

```
1 #include "MyEvent.h"
2
3 ClassImp(MyEvent)
4
5 MyEvent::MyEvent() : fRunNumber(0), fEventNumber(0),
6                     fLuminosityBlock(0) {
7     // Construtor padrão
8 }
9
10 void MyEvent::AddMuon(const MyParticle& mu) {
11     fMuons.push_back(mu);
12 }
13
14 void MyEvent::AddElectron(const MyParticle& el) {
15     fElectrons.push_back(el);
16 }
```

Exemplo Físico: Classe MyEvent (Implementação)

```
1 void MyEvent::AddJet(const MyParticle& jet) {  
2     fJets.push_back(jet);  
3 }
```

Listing 15: MyEvent.cc

Exemplo: LinkDef.h

```
1 #ifdef __CINT__
2 #pragma link off all globals;
3 #pragma link off all classes;
4 #pragma link off all functions;
5
6 #pragma link C++ class MyParticle+;
7 #pragma link C++ class vector<MyParticle>;
8 #pragma link C++ class MyEvent+;
9 #endif
```

Listing 16: LinkDef.h

Exemplo: Makefile Completo

```
1 CXX = g++
2 CXXFLAGS = -O2 -Wall -fPIC
3 ROOTCFLAGS = $(shell root-config --cflags)
4 ROOTLIBS = $(shell root-config --glibs)
5 INCLUDES = -I$(ROOTSYS)/include
6
7 OBJS = MyParticle.o MyEvent.o eventdict.o
8
9 all: libMyEvent.so
10
11 eventdict.cc: MyParticle.h MyEvent.h LinkDef.h
12     rootcint -f $@ -c $(INCLUDES) $^
13
14 %.o: %.cc
15     $(CXX) $(CXXFLAGS) $(ROOTCFLAGS) $(INCLUDES) -c $<
```

Exemplo: Makefile Completo

```
1 libMyEvent.so: $(OBJS)
2     $(CXX) -shared -o $@ $^ $(ROOTLIBS)
3
4 clean:
5     rm -f *.o *.so eventdict.*
```

Listing 17: Makefile

Usando a Biblioteca no ROOT

```
1 root [] .L libMyEvent.so
2 root [] MyEvent *ev = new MyEvent()
3 root [] MyParticle mu(100.0, 10.0, 5.0, 1.0) // E=100, px=10, py=5,
      pz=1
4 root [] ev->AddMuon(mu)
5 root [] ev->GetEventNumber()
```

Listing 18: Carregando e usando a biblioteca

Arquivo eventdict.cc - O que é gerado?

O rootcint gera um arquivo que contém:

- Implementação dos métodos de serialização
- Dicionário para o interpretador CINT
- Tabela de funções para acesso via linha de comando

```
1 // Streamer para MyParticle
2 void MyParticle::Streamer(TBuffer &b) {
3     if (b.IsReading()) {
4         // Lê os dados do buffer
5     } else {
6         // Escreve os dados no buffer
7     }
8 }
```

Arquivo eventdict.cc - O que é gerado?

```
1 // Função para ShowMembers
2 void MyParticle::ShowMembers(TMemberInspector &insp) {
3     // Mostra membros para o interpretador
4 }
```

Listing 19: Exemplo do conteúdo gerado

```
1 root [] gROOT->Macro("MyEvent.C++")
```

Listing 20: Compilando com ACLiC

- Compila automaticamente
- Não precisa de Makefile ou LinkDef.h
- Útil para desenvolvimento rápido
- Menos controle sobre o processo

- Python permite estender suas funcionalidades com módulos em C/C++
- Útil para:
 - Acelerar partes críticas do código
 - Reutilizar bibliotecas existentes em C++
 - Integrar com frameworks de física (ex: ROOT)
- API Python fornece funções, macros e variáveis para acesso ao sistema
- Para usar: instalar `python-dev`

Exemplo: Módulo C++ para Python

```
1 #include <Python.h>
2 #include <iostream>
3 // Função que será chamada do Python
4 static PyObject* testSrc(PyObject* self, PyObject* args) {
5     const char* name;
6     // Parseia argumentos: espera uma string
7     if (!PyArg_ParseTuple(args, "s", &name))
8         return NULL;
9     // Faz algo com o argumento
10    std::cout << "Hello from C++: " << name << std::endl;
11    // Retorna None (não retorna valor)
12    Py_RETURN_NONE;
13 }
```

Exemplo: Módulo C++ para Python

```
1 // Tabela de métodos do módulo
2 static PyMethodDef MyMethods[] = {
3     {"my_test", testSrc, METH_VARARGS, "Print a greeting."},
4     {NULL, NULL, 0, NULL} // Sentinela
5 };
6 // Definição do módulo
7 static struct PyModuleDef MyModule = {
8     PyModuleDef_HEAD_INIT,
9     "MyModule", // Nome do módulo
10    NULL, // Documentação
11    -1, // Tamanho do estado
12    MyMethods // Métodos
13 };
14 // Função de inicialização - NOME DEVE SER "init" + NomeDoModulo
15 PyMODINIT_FUNC PyInit_MyModule(void) {
16     return PyModule_Create(&MyModule);
17 }
```

Explicação da API Python/C

- PyObject*: Tipo base para todos os objetos Python
- self: Para funções de módulo, aponta para o módulo
- args: Tupla com argumentos passados do Python
- PyArg_ParseTuple(): Converte argumentos Python para C

```
1 "s" - string (const char*)
2 "i" - inteiro (int)
3 "d" - double (double)
4 "f" - float (float)
5 "O" - objeto Python (PyObject*)
```

Listing 22: Formatos de PyArgParseTuple

- Retorno: Deve ser um objeto Python
- Para retornar valores: use Py_BuildValue()
- Para não retornar: Py_RETURN_NONE

Retornando Valores do C++ para Python

```
1 static PyObject* addNumbers(PyObject* self, PyObject* args) {
2     double a, b;
3     if (!PyArg_ParseTuple(args, "dd", &a, &b))
4         return NULL;
5     double result = a + b;
6
7     // Converte double para objeto Python
8     return Py_BuildValue("d", result);
9 }
10 // Na tabela de métodos:
11 static PyMethodDef MyMethods[] = {
12     {"add", addNumbers, METH_VARARGS, "Add two numbers."},
13     {"my_test", testSrc, METH_VARARGS, "Print a greeting."},
14     {NULL, NULL, 0, NULL}
15 };
```

Listing 23: Função que retorna um valor

Compilando o Módulo Python

```
1 from distutils.core import setup, Extension
2 # Define o módulo
3 module1 = Extension('MyModule',
4                     sources = ['testmodule.cc'])
5 setup(name = 'MyPackage',
6       version = '1.0',
7       description = 'This is a demo package',
8       ext_modules = [module1])
```

Listing 25: setup.py - Script de compilação

```
1 python setup.py build
2 # Instalar no diretório pessoal
3 pip install --target=$HOME/python .
4 # OU instalar no sistema (requer sudo)
5 sudo python setup.py install
```

Listing 26: Comandos para compilar

Usando o Módulo no Python

```
1 #!/usr/bin/env python
2 import sys
3 import os
4
5 # Adiciona o diretório de instalação ao path
6 home = os.environ['HOME']
7 mypythonpath = os.path.join(home, "python/lib/python")
8 sys.path.append(mypythonpath)
9
10 # Importa o módulo
11 import MyModule
12
13 # Usa as funções
14 MyModule.my_test("Hello World")
15
16 result = MyModule.add(10.5, 20.3)
17 print(f"Result: {result}")
```

- PyROOT é um módulo de extensão Python
- Permite interagir com qualquer classe ROOT do interpretador Python
- Fornece bindings Python para ROOT
- Permite chamadas cruzadas entre ROOT/CINT e Python
- Transporte de objetos entre interpretadores

Para trabalhar com PyROOT, a variável PYTHONPATH precisa ser configurada (no ambiente snap):

```
1 export ROOTSYS=/snap/root-framework/current/usr/local
2 source $ROOTSYS/bin/thisroot.sh
3 python3  pytest.py
4 #ou
5 pyroot  pytest.py
```

Importando PyROOT

```
1 # Importa o módulo principal
2 import ROOT
3 # OU importa módulos específicos
4 from ROOT import TCanvas
5 from ROOT import TGraph, TH2F
6 # OU importa tudo (não recomendado)
7 from ROOT import *
8
9 # Regra geral: ":" no CINT vira "." no PyROOT
10 # CINT: canvas = new TCanvas("canvas","",500,500)
11 # PyROOT:
12 canvas = ROOT.TCanvas("canvas", "", 500, 500)
```

Listing 29: Importando PyROOT

Exemplo 1: Plotando um TGraph

```
1 #!/usr/bin/env python
2 import ROOT
3 from array import array
4 # Desativa plotagem na tela (modo batch)
5 ROOT.gROOT.SetBatch(True)
6 def main():
7     # Cria arrays para os dados
8     x = array("d")
9     y = array("d")
10    x.append(1.0)
11    y.append(1.0)
12    x.append(3.0)
13    y.append(2.0)
14    x.append(4.0)
15    y.append(2.5)
16    x.append(6.0)
17    y.append(3.0)
```

Exemplo 1: Plotando um TGraph

```
1  n = len(x)
2
3  # Cria canvas
4  canvas = ROOT.TCanvas("someName", "Graph Plot", 500, 500)
5  canvas.SetFillColor(0)
6  canvas.cd()
7
8  # Cria frame para eixos
9  frame = ROOT.TH2F("frame", "", 2, 0, 6, 2, 0, 4)
10 frame.SetStats(0)
11 frame.GetXaxis().SetTitle("x")
12 frame.GetYaxis().SetTitle("y")
13 frame.Draw()
```

Exemplo 1: Plotando um TGraph

```
1  # Cria e plota o gráfico
2  graph = ROOT.TGraph(n, x, y)
3  graph.SetMarkerStyle(2)      # Quadrado
4  graph.SetMarkerSize(2)      # Tamanho 2
5  graph.SetLineColor(2)       # Vermelho
6  graph.Draw("PL")            # Pontos e linhas
7
8  # Salva a figura
9  canvas.Print("graph.eps")
10 print("Graph saved to graph.eps")
11
12 if __name__ == "__main__":
13     main()
```

Listing 30: graph.py - Plotando com PyROOT

Exemplo 2: Selecionando Eventos de uma Tree

- Útil para selecionar subconjuntos de eventos para análise detalhada
- Seleção baseada em: run number, luminosity block, event number
- Formato do arquivo: run:lumi:event

```
1  #!/usr/bin/env python3
2  import sys, os, re
3  import ROOT
4  root_re = re.compile(r"(?P<rootfile>([~/]*)\.root$)")
5  event_re = re.compile(r"(?P<run>(\d+)):(?P<lumi>(\d+)):(?P<event>(\d
6  +))")
7  def main():
8      if len(sys.argv) == 1:
9          print()
10         print("# Usage: " + sys.argv[0] + " <root file> -pick <pick
11         events file>")
12         print()
13         sys.exit()
```

Exemplo 2: Selecionando Eventos de uma Tree

```
1 rootfiles = []
2 pickeventsfile = ""
3 iarg = 1
4 while iarg < len(sys.argv):
5     if sys.argv[iarg] == "-pick" and iarg < len(sys.argv) - 1:
6         pickeventsfile = sys.argv[iarg + 1]
7         iarg += 1
8     match = root_re.search(sys.argv[iarg])
9     if match:
10         rootfiles.append(sys.argv[iarg])
11     iarg += 1
12
13 events = getEvents(pickeventsfile)
14 for file in rootfiles:
15     pick(file, events)
```

Exemplo 2: Funções Auxiliares

```
1 def getEvents(filename):
2     events = []
3     fIN = open(filename, 'r')
4     for line in fIN:
5         events.append(line.replace("\n", ""))
6     return events
7
8
9 def pick(filename, events):
10    fIN = ROOT.TFile.Open(filename)
11    fName = "picked.root"
12    match = root_re.search(filename)
13    if match:
14        namebody = match.group("rootfile")
15        fName = filename.replace(namebody, "picked_" + namebody)
```

Exemplo 2: Funções Auxiliares

```
1 fOUT = ROOT.TFile.Open(fName, 'RECREATE')
2 intree = fIN.Get("Events") //Nome da arvore pré existente
3 tree = intree.CloneTree(0)
4
5 for event in events:
6     match = event_re.search(event)
7     if match:
8         selection = "run==" + match.group("run")
9         selection += "&&lumi==" + match.group("lumi")
10        selection += "&&event==" + match.group("event")
11        picktree = intree.CopyTree(selection)
12        treelist = ROOT.TList()
13        treelist.Add(picktree)
14        tree.Merge(treelist)
```

Exemplo 2: Funções Auxiliares

```
1 print("Saving", tree.GetEntries(), "events")
2 tree.AutoSave()
3 fIN.Close()
4 fOUT.Close()
5
6
7 if __name__ == "__main__":
8     main()
```

Listing 32: pickEvents.py

Usando a Classe Rootificada no Python

```
1 #!/usr/bin/env python
2 import ROOT
3
4 # Carrega a biblioteca com as classes
5 ROOT.gSystem.Load("libMyEvent.so")
6
7 # Cria objetos como no C++
8 muon = ROOT.MyParticle(100.0, 10.0, 5.0, 1.0, -1, 13)
9 muon.Print()
10
11 # Cria evento
12 event = ROOT.MyEvent(1, 12345, 50)
13 event.AddMuon(muon)
14 event.AddElectron(ROOT.MyParticle(50.0, 5.0, 2.0, 0.5, -1, 11))
15
16 print(f"Number of muons: {event.GetNMuons()}")
17 print(f"Number of electrons: {event.GetNElectrons()}")
```

Usando a Classe Rootificada no Python

```
1 # Salva em arquivo
2 f = ROOT.TFile("events.root", "RECREATE")
3 event.Write("event1")
4 f.Close()
5
6 # Lê de volta
7 f2 = ROOT.TFile("events.root")
8 event2 = f2.Get("event1")
9 event2.PrintSummary()
10 f2.Close()
```

Listing 34: Usando classes ROOT no Python

- Exemplos oficiais do ROOT:

1 `$ROOTSYS/tutorials/pyroot/`

- Exemplos incluídos:

- `fillrandom.py` - Preenchendo histogramas
- `fit.py` - Ajuste de funções
- `tree.py` - Trabalhando com TTrees
- `graph.py` - Plotando gráficos